

format () Method

This method is used to format strings. The string on which this method is called can contain literal text or replacement fields delimited by braces {}. Each replacement field contains either the numeric index of a positional argument, or the name of a keyword argument. It returns a copy of the string where each replacement field is replaced with the string value of the corresponding argument.

Syntax:- `str.format(arg)`

Ex:-

```
str = "My age is {}"
```

```
print(str.format(62))
```

Ex:- `print("My age is {}".format(62))`

In most of the cases the syntax is similar to the old %-formatting, with the addition of the {} and with : used instead of %.

Ex:-

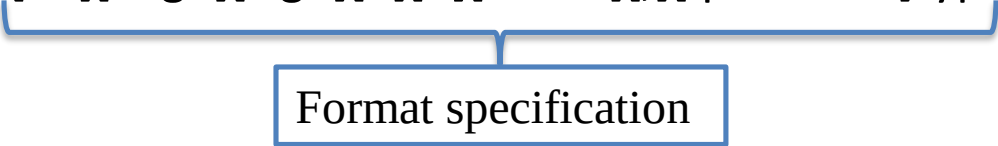
'%d' can be translated to '{:d}'

'%05.3f' can be translated to '{:05.3f}'.

format () Method

```
print("Replacement Field".format(values))
```

```
RF = {index/key:[fill][align][sign][#][0][width][,][.precision]type}
```



Format specification

Ex:-

```
print("{}".format(10))
```

```
print("{} {}".format(10, 20))
```

```
print("Mobile Price{} Computer Price {}".format(10, 20))
```

Format Method

Conversion Type	Meaning
d	Signed integer decimal.
o	Signed octal value.
x	Signed hexadecimal (lowercase).
X	Signed hexadecimal (uppercase).
b	Binary Format
e	Floating point exponential format (lowercase).
E	Floating point exponential format (uppercase).
f	Floating point decimal format. (Default: 6)
F	Same as 'f'. Except displays 'inf' as 'INF' and 'nan' as 'NAN'

Format Method

Conversion Type	Meaning
c	Character. Converts the integer to the corresponding Unicode character before printing
g	General format. Rounds number to p significant digits. (Default precision: 6)
G	Same as 'g'. Except switches to 'E' if the number is large.
n	Same as 'd'. Except it uses current locale setting for number separator
s	String (converts any Python object using str()).
%	Percentage. Multiplies the number by 100 and displays in fixed ('f') format, followed by a percent sign.

Format Method

Alignment Type	Meaning
<	Forces the field to be left-aligned within the available space (This is default for most objects)
^	Forces the field to be centered within the available space.
>	Forces the field to be right-aligned within the available space (This is default for Numbers)
=	Forces the padding to be placed after the sign (if any) but before the digits. This is used for printing fields in the form '+000000120'. This alignment option is only valid for numeric types. It becomes the default when '0' immediately precedes the field width.

Format Method

Sign	Meaning
+	indicates that a sign should be used for both positive as well as negative numbers.
-	indicates that a sign should be used only for negative numbers (this is the default behavior).
	(a space) indicates that a leading space should be used on positive numbers, and a minus sign on negative numbers.

Format Method

- The '#' option causes the “alternate form” to be used for the conversion. The alternate form is defined differently for different types. This option is only valid for integer, float, complex and Decimal types. For integers, when binary, octal, or hexadecimal output is used, this option adds the prefix respective '0b', '0o', or '0x' to the output value.
- The ',' option signals the use of a comma for a thousands separator. For a locale aware separator, use the 'n' integer presentation type instead.
- The '_' option signals the use of an underscore for a thousands separator for floating point presentation types and for integer presentation type 'd'.

Format Method

<code>print("{}".format(10))</code>	10
<code>print("{} {}".format(10, 20))</code>	10 20
<code>print("{0}".format(10))</code>	10
<code>print("{0} {1}".format(10, 20))</code>	10 20
<code>print("{1} {0}".format(10, 20))</code>	20 10
<code>print("{num1}".format(num1=10))</code>	10
<code>print("{num1} {num2}".format(num1=10, num2=20))</code>	10 20
<code>print("{num2} {num1}".format(num1=10, num2=20))</code>	20 10

Format Method

```
print("{}".format(10.56))
```

```
print("{} {}".format(10.56, 20.42))
```

```
print("{0}".format(10.56))
```

```
print("{0} {1}".format(10.56, 20.42))
```

```
print("{1} {0}".format(10.56, 20.42))
```

```
print("{num1}".format(num1=10.56))
```

```
print("{num1} {num2}".format(num1=10.56, num2=20.42))
```

```
print("{num2} {num1}".format(num1=10.56, num2=20.42))
```

Format Method

```
print("{}".format("GeekyShows"))
```

GeekyShows

```
print("{} {}".format("Geeky", "Shows"))
```

Geeky Shows

```
print("{0}".format("Geeky"))
```

Geeky

```
print("{0} {1}".format("Geeky", "Shows"))
```

Geeky Shows

```
print("{1} {0}".format("Geeky", "Shows"))
```

Shows Geeky

```
print("{str1}".format(str1="GeekyShows"))
```

GeekyShows

```
print("{str1} {str2}".format(str1="Geeky", str2="Shows"))
```

Geeky Shows

```
print("{str2} {str1}".format(str1="Geeky", str2="Shows"))
```

Shows Geeky

Format Method

```
print("Hello My Name is {}".format("GeekyShows"))
```

```
print("{} {}".format(10, "Shows"))
```

```
print("{0} {1}".format(10, "Shows"))
```

```
print("{1} {0}".format(10, "Shows"))
```

```
print("{num1} {str1}".format(num1=10, str1="Shows"))
```

```
print("{str1} {num1}".format(num1=10, str1="Shows"))
```

Format Method

```
print("{}".format(15))
```

```
print("{:d}".format(15))
```

```
print("{0:d}".format(15))
```

```
print("{num1:d}".format(num1=15))
```

Format Method

:`[fill][align][sign][#][0][width][,][.precision]`type

```
print("{num:5d}".format(num=15))
```

			1	5
--	--	--	---	---

```
print("{num:*<5d}".format(num=15))
```

1	5	*	*	*
---	---	---	---	---

```
print("{num:05d}".format(num=15))
```

0	0	0	1	5
---	---	---	---	---

```
print("{num:*>5d}".format(num=15))
```

*	*	*	1	5
---	---	---	---	---

```
print("{num:+5d}".format(num=15))
```

		+	1	5
--	--	---	---	---

```
print("{num:^5d}".format(num=15))
```

	1	5		
--	---	---	--	--

```
print("{num:<5d}".format(num=15))
```

1	5			
---	---	--	--	--

Format Method

:`[fill][align][sign][#][0][width][,][.precision]`type

`print("{num:8f}".format(num=15.65))`

1	5	.	6	5	0	0	0	0
---	---	---	---	---	---	---	---	---

`print("{num:*<8.2f}".format(num=15.65))`

1	5	.	6	5	*	*	*
---	---	---	---	---	---	---	---

`print("{num:8.3f}".format(num=15.65))`

		1	5	.	6	5	0
--	--	---	---	---	---	---	---

`print("{num:*>8.2f}".format(num=15.65))`

*	*	*	1	5	.	6	5
---	---	---	---	---	---	---	---

`print("{num:+8.2f}".format(num=15.65))`

		+	1	5	.	6	5
--	--	---	---	---	---	---	---

`print("{num:^8.2f}".format(num=15.65))`

	1	5	.	6	5		
--	---	---	---	---	---	--	--

`print("{num:<8.2f}".format(num=15.65))`

1	5	.	6	5			
---	---	---	---	---	--	--	--

Format Method

:`[fill][align][sign][#][0][width][,][.precision]`type

```
print("{:8s}".format("Geek"))
```

G	e	e	k				
---	---	---	---	--	--	--	--

```
print("{:>8s}".format("Geek"))
```

				G	e	e	k
--	--	--	--	---	---	---	---

```
print("{:<8s}".format("Geek"))
```

G	e	e	k				
---	---	---	---	--	--	--	--

```
print("{:*>8s}".format("Geek"))
```

*	*	*	*	G	e	e	k
---	---	---	---	---	---	---	---

```
print("{:*<8s}".format("Geek"))
```

G	e	e	k	*	*	*	*
---	---	---	---	---	---	---	---

```
print("{:^8s}".format("Geek"))
```

		G	e	e	k		
--	--	---	---	---	---	--	--

Format Method

:
[fill][align][sign][#][0][width][,][.precision]type

```
print("{:.3s}".format("GeekShows"))
```

G	e	e
---	---	---

```
print("{:>8.3}".format("GeekShows"))
```

					G	e	e
--	--	--	--	--	---	---	---

```
print("{:8.3}".format("GeekShows"))
```

G	e	e					
---	---	---	--	--	--	--	--

```
print("{:*>8.3s}".format("GeekShows"))
```

*	*	*	*	*	G	e	e
---	---	---	---	---	---	---	---

```
print("{:*<8.3}".format("GeekyShows"))
```

G	e	e	*	*	*	*	*
---	---	---	---	---	---	---	---

```
print("{:^8.3s}".format("GeekShows"))
```

		G	e	e			
--	--	---	---	---	--	--	--